Design and Implementation of a Hardware Accelerated Matrix Multiplier

1 Design Trade-off Analysis

To address the latency and throughput limitations of CPU-based execution, we propose a custom hardware accelerator. The design choice is critical: it must balance computational speed with available FPGA resources (DSP slices, BRAM, and LUTs). We evaluated two primary architectural approaches: a Full Systolic Array and a Tiled (Blocked) Architecture.

1.1 Approach 1: Full Systolic Array

A systolic array consists of a 2D grid of Processing Elements (PEs) that perform multiply-accumulate (MAC) operations while data flows rhythmically across the array.

For an $N \times N$ matrix multiplication, a full systolic array requires:

$$N^2 \text{ Processing Elements}$$

Each PE performs one multiplication and accumulation per clock cycle.
Complexity Order = $O(N)$ latency

1.2 Approach 2: Tiled (Blocked) Architecture

The tiled architecture divides matrices into smaller sub-blocks (tiles) and processes them sequentially using a limited number of hardware resources.

Instead of computing the full matrix at once, the same processing hardware is reused multiple times to complete the overall multiplication. Each tile performs partial multiplication and accumulation. Results from all tiles are combined to obtain the final matrix.

Resource and Complexity Analysis:
Consider an $N \times N$ matrix where the tile size is defined as $B \times B$.

- **Number of Tiles:** The total tiles required to cover the output space is:

$$\text{Total Tiles} = \frac{N^2}{B^2}$$

- **Processing Elements (PE):** Assuming one PE per tile to maximize throughput within the block:

$$\text{PE Count} = \frac{N^2}{B^2}$$

- **DSP Utilization:** Each PE requires at least one DSP slice for the MAC operation. Assuming 1 DSP per MAC:

$$\text{DSP Count} = \frac{N^2}{B^2} \text{ slices}$$

- **Computational Latency:** While the total operations remain $O(N^3)$, the sequential nature of tiling results in a latency order of:

$$\text{Latency} \approx O\left(\frac{N^3}{Nofblocks}\right)$$

$$\approx O\left(\frac{N^3}{\left(\frac{N^2}{B^2}\right)}\right)$$

$$\approx O\left(N \times B^2\right)$$

1.3 Conclusion on Architecture Selection

After evaluating both architectural approaches, the Full Systolic Array is selected as the final design choice. Since the primary objective of this project is to maximize performance and minimize execution time, the systolic array provides the best solution despite its higher resource usage.

2 Traditional Systolic Array Architecture

The architecture is a 2D mesh of Processing Elements (PEs). For an $n \times n$ matrix multiplication, the array consists of n^2 PEs arranged in a grid.

Each node in the grid ($PE_{i,j}$ where $0 \leq i, j < n$) is a fundamental computation unit. Each PE is primarily tasked with performing multiply–accumulate computations. Additionally, each PE also requires interconnection circuits for data transmission.

In every clock cycle, a PE performs the following logic:

1. Receives an element from matrix A (from the left).
2. Receives an element from matrix B (from the top).
3. Multiplies them and adds the result to a local accumulator (partial sum).
4. Passes the data A to the right neighbor and data B to the bottom neighbor.

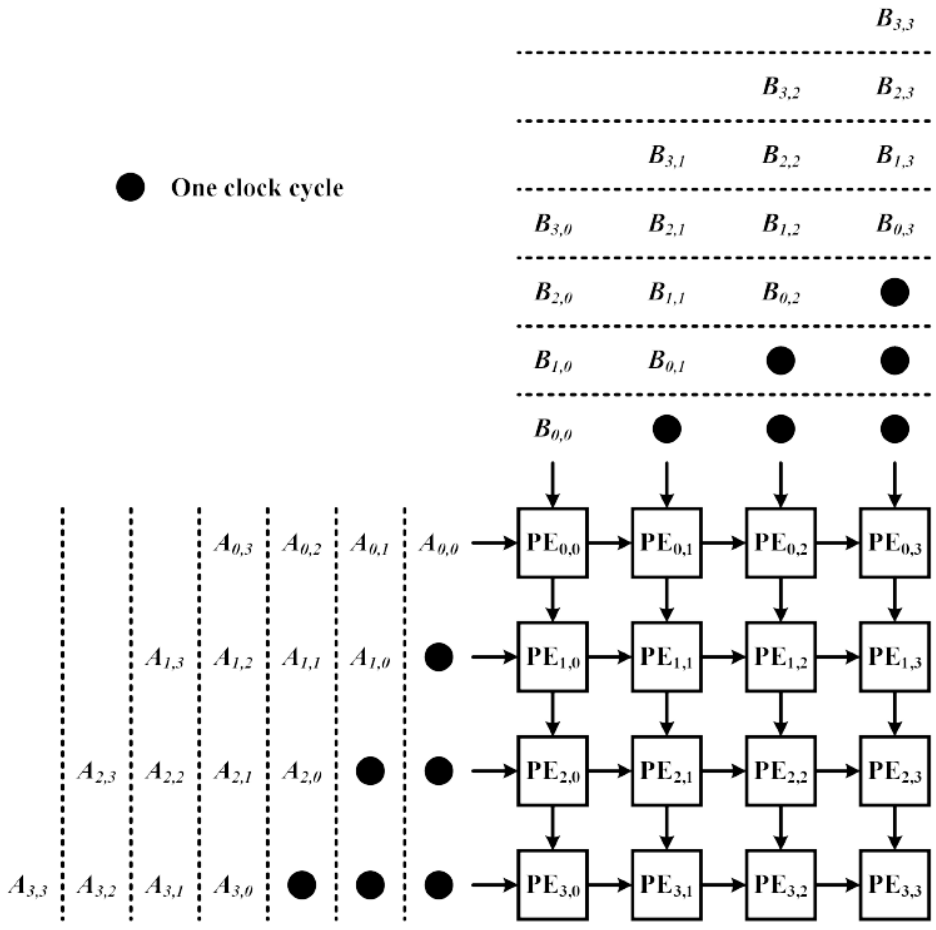


Figure 1: 4 x 4 systolic array architecture.

2.1 Methodology and Data Flow

To ensure the correct elements meet at the correct PE at the exact right time, the input data must be "skewed" (delayed) before entering the array. This is a critical aspect.

2.1.1 Input Skewing

As shown in the architectural diagram, data is not fed into the array simultaneously for all rows and columns.

- **Matrix A (Row Input):** Enters from the left. Row i is delayed by i clock cycles.
  - Row 0: No delay.
  - Row 1: 1 cycle delay.
  - Row 2: 2 cycle delay, etc.
- **Matrix B (Column Input):** Enters from the top. Column j is delayed by j clock cycles.
  - Column 0: No delay.
  - Column 1: 1 cycle delay, etc.

2.1.2 Execution Flow

Taking a 4×4 array as an example:

1. **Cycle 1:** $PE_{0,0}$ computes $A_{0,0} \times B_{0,0}$.
2. **Cycle 2:**
   - $A_{0,0}$ moves to $PE_{0,1}$ to meet $B_{1,1}$ (due to B's skew).
   - $B_{0,0}$ moves to $PE_{1,0}$ to meet $A_{1,0}$ (due to A's skew).
   - $PE_{0,0}$ receives the next inputs ($A_{0,1}$ and $B_{1,0}$).

3 Our proposed Architecture Implementation

3.1 Cannon's Algorithm

Cannon's Algorithm improves efficiency by replacing the "wave" method with a "shift-and-compute" method. This keeps all PEs active for a longer duration.

3.1.1 Architecture Modification

As shown in the figure 2, the PEs are connected in a torus topology:

- The output of the last PE in a row connects back to the first PE in that same row.
- The output of the last PE in a column connects back to the first PE in that same column.

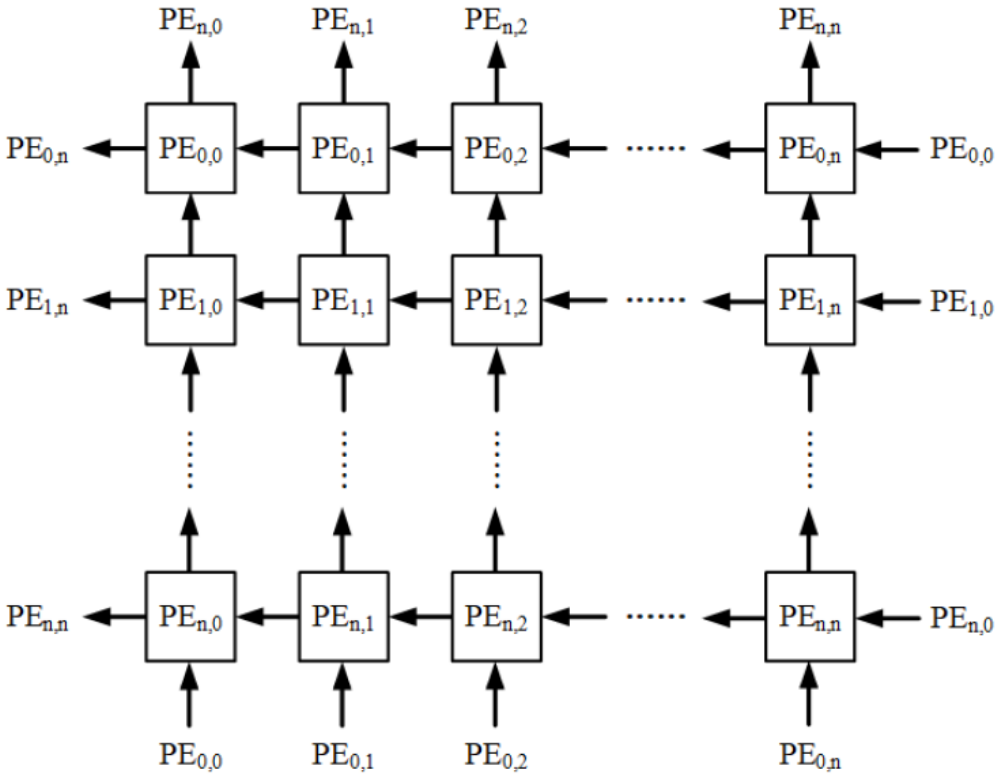


Figure 2: systolic array architecture applied to Cannon's algorithm.

3.1.2 Step-by-Step Methodology

Cannon's algorithm executes in three main phases:

Step 1: Initial Alignment

Before computation starts, the matrices are shifted to align the data correctly:

- **Matrix A (Rows):** Row i is shifted left by i steps. (Row 0 shifts 0, Row 1 shifts 1, etc.)
- **Matrix B (Columns):** Column j is shifted upward by j steps.

Matrix A and B before alignment:

$$A = \begin{bmatrix} A_{00} & A_{01} & A_{02} & A_{03} \\ A_{10} & A_{11} & A_{12} & A_{13} \\ A_{20} & A_{21} & A_{22} & A_{23} \\ A_{30} & A_{31} & A_{32} & A_{33} \end{bmatrix} \quad B = \begin{bmatrix} B_{00} & B_{01} & B_{02} & B_{03} \\ B_{10} & B_{11} & B_{12} & B_{13} \\ B_{20} & B_{21} & B_{22} & B_{23} \\ B_{30} & B_{31} & B_{32} & B_{33} \end{bmatrix}$$

Aligned matrix A and B becomes:

$$A_{\text{aligned}} = \begin{bmatrix} A_{00} & A_{01} & A_{02} & A_{03} \\ A_{11} & A_{12} & A_{13} & A_{10} \\ A_{22} & A_{23} & A_{20} & A_{21} \\ A_{33} & A_{30} & A_{31} & A_{32} \end{bmatrix} \quad B_{\text{aligned}} = \begin{bmatrix} B_{00} & B_{11} & B_{22} & B_{33} \\ B_{10} & B_{21} & B_{32} & B_{03} \\ B_{20} & B_{31} & B_{02} & B_{13} \\ B_{30} & B_{01} & B_{12} & B_{23} \end{bmatrix}$$

Step 2: Multiplication and Accumulation

Once aligned, every PE performs a multiply-accumulate operation simultaneously.

$$C_{i,j} = C_{i,j} + (A_{local} \times B_{local})$$

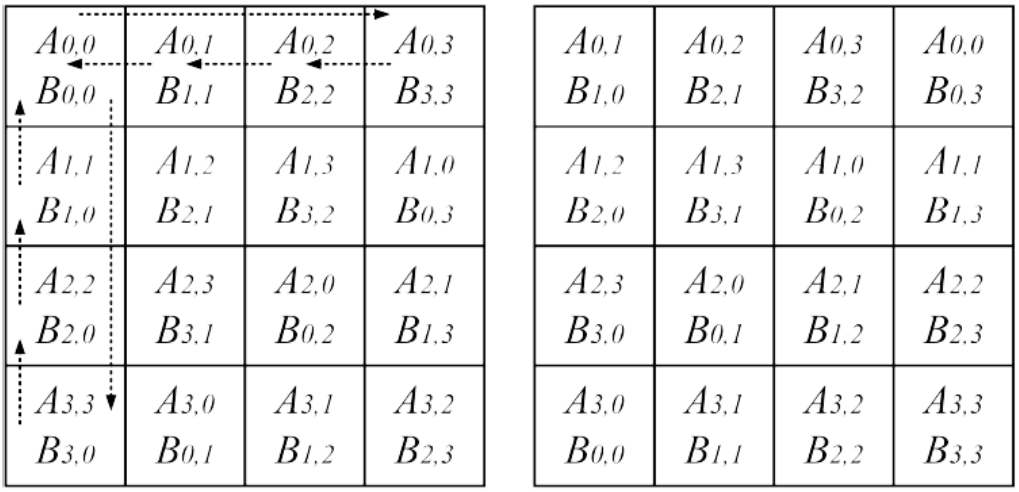


Figure 3: (a) The overlap of aligned matrices A and B. (b) The results after the first global matrix shifting.

Step 3: Global Matrix Shifting

After the multiplication, the data moves for the next round:

- All elements of Matrix A shift one step to the **left** (wrapping around to the right if needed).
- All elements of Matrix B shift one step **up** (wrapping around to the bottom if needed).

In Cannon's algorithm, step 2 must be executed a total of n times, and step 3 must be executed n-1 times in total. Then, the resulting matrix C is obtained.
only n clock cycles are needed to complete the matrix multiplication. Taking a 4×4 systolic array as an example, if matrix operations are performed using the traditional algorithm, it would require 10 clock cycles, whereas utilizing Cannon's algorithm for matrix operations would only require four clock cycles.

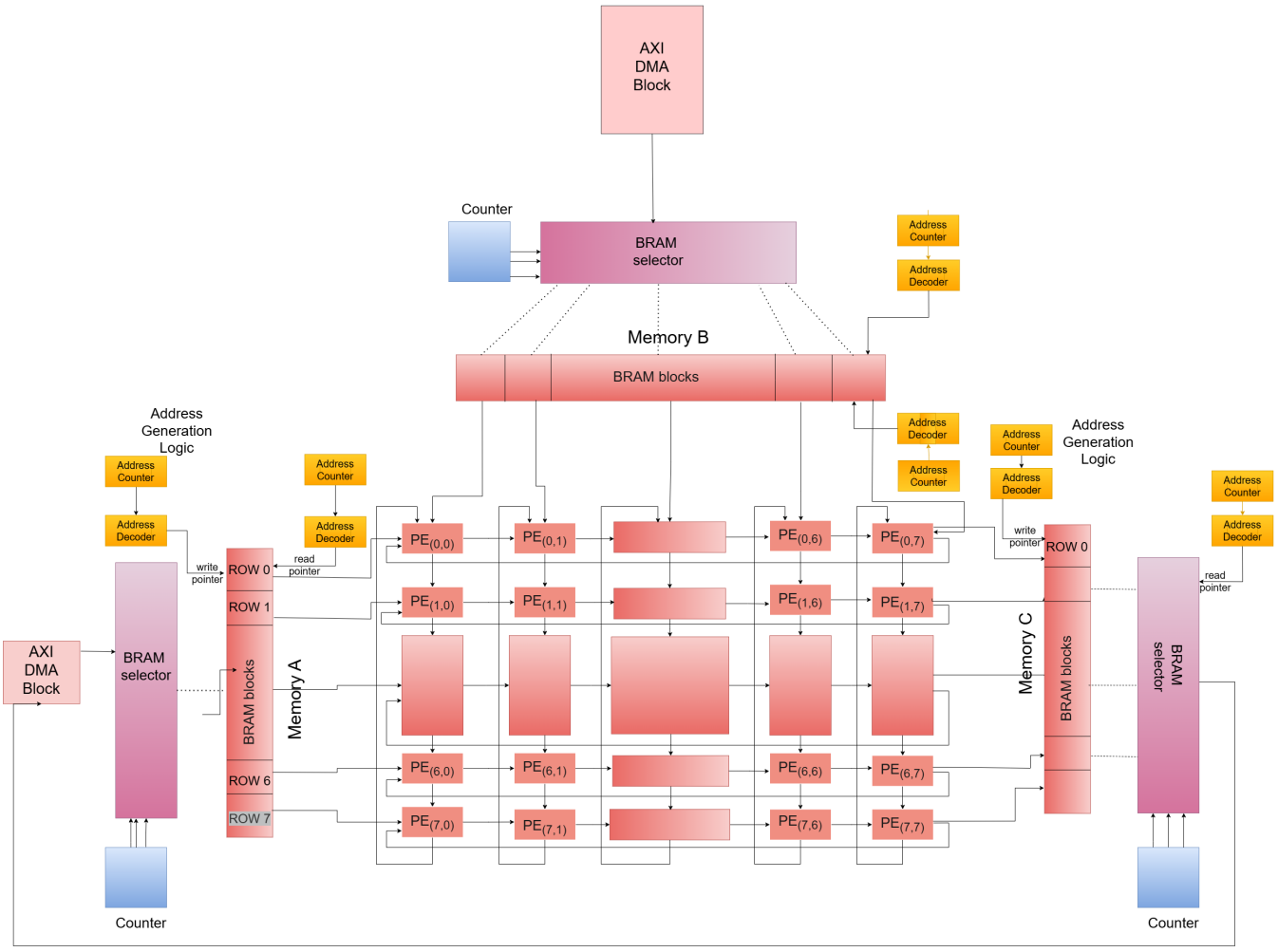


Figure 4: Micro architecture.

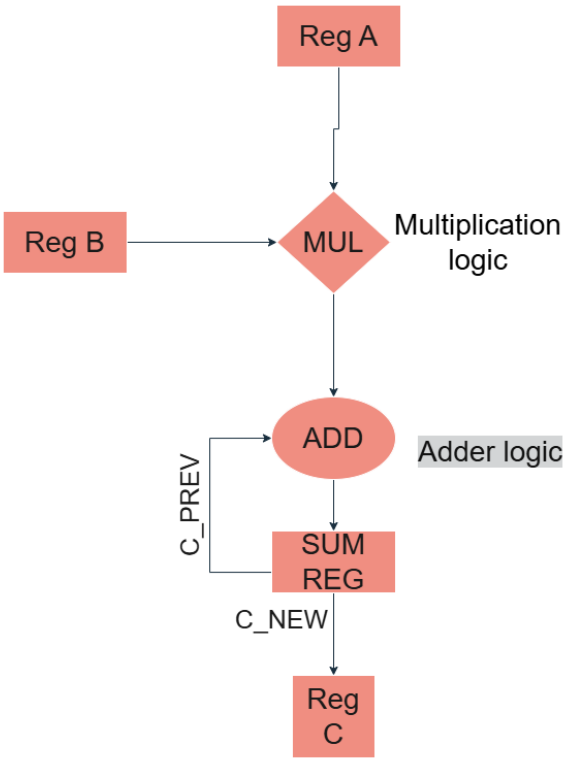


Figure 5: MAC operation inside PE

4 Algorithm Implementation

The following pseudocode describes the control logic.

Algorithm 1 Cannon's Matrix Multiplication (8×8)

Require: Matrices A, B (Pre-loaded in BRAM)

Ensure: Matrix C (Result)

Step 1: Alignment
for $i \leftarrow 0$ to 7 **do**
    Shift Row $A[i]$ Left by i
    Shift Col $B[i]$ Up by i
end for
Step 2: Processing Loop
for $step \leftarrow 0$ to 7 **do**
    // Parallel Execution in all 64 PEs
    for each $PE_{i,j}$ in parallel **do**
        $PE_{i,j}.Sum \leftarrow PE_{i,j}.Sum + (PE_{i,j}.A \times PE_{i,j}.B)$
        // Prepare for next step
        $PE_{i,j}.nextA \leftarrow PE_{i,j+1}.A$ *// Receive from East*
        $PE_{i,j}.nextB \leftarrow PE_{i+1,j}.B$*// Receive from South*
    end for
    Update all PEs with $nextA$ and $nextB$
end for
Step 3: Output
Stream $PE.Sum$ to Output BRAM Matrix C buffer

5 References

- AMD reference documet for DSP48E2
- Wikipedia reference for Systolic Array
- Hsin-Chen Lu, Liang-Ying Su and Shih-Hsu Huang: Highly Fault-Tolerant Systolic-Array-Based Matrix Multiplication.
- Zhaoqin "Richie" Li, Sicheng Chen:Design and Implementation of an FPGA-Based Tiled Matrix Multiplication Accelerator for Transformer Self-Attention on the Xilinx KV260 SoM.